SPIDER-MAN 2D — GUÍA COMPLETA DEL PROYECTO, TECNOLOGÍAS Y SPRITES

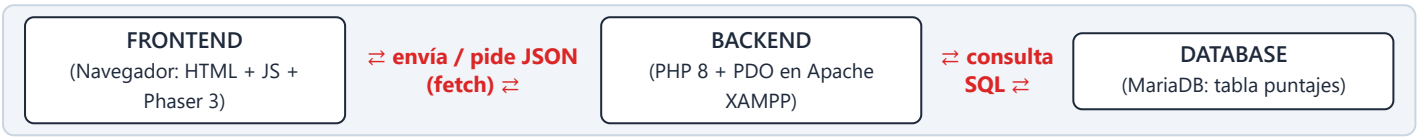
Documento unificado: Arquitectura, Workflow paso a paso, Archivos, Procesamiento de Sprites con Python y Mecánicas en Phaser 3

1. ¿CÓMO ESTÁ DIVIDIDO EL PROYECTO? (ARQUITECTURA EN 3 CARPETAS)

1. Carpeta frontend/
Es lo que ve y juega el usuario. Contiene la página web, el diseño estilo cómic, el código del juego en Phaser 3 y las imágenes PNG de los personajes y la ciudad.

2. Carpeta backend/
Es el puente o mensajero (PHP). Recibe los pedidos del navegador (pedir el Top 20 o guardar un puntaje nuevo), valida los datos y se comunica con la base de datos.

3. Carpeta database/
Es donde se guardan los datos (SQL). Contiene el script que crea la base de datos `spiderman_db` y la tabla `puntajes` en el motor MariaDB de XAMPP.



2. EL WORKFLOW DE LA PÁGINA (¿QUÉ PASA PASO A PASO?)

1. **Abrís la página en el navegador:** Se carga `frontend/index.html` junto con `style.css`. El juego arranca en **pausa** detrás de una ventana superpuesta que muestra los controles y la **Tabla de Posiciones**.
2. **Se cargan los puntajes de la Base de Datos:** Apenas carga la página, `main.js` usa `fetch()` para pedirle a `backend/api/puntajes/index.php` el Top 20 guardado en MariaDB, ordenado primero por **más puntos** y, ante empate, por **menor tiempo**.
3. **Precarga de Sprites (Preload):** Mientras mirás la tabla, Phaser lee el catálogo de `assets_data.js` y carga en memoria las **440 imágenes PNG** de enemigos y jefes más los sprites de Spider-Man para que el juego funcione fluido sin trabarse.
4. **Empezás a jugar:** Al pulsar "*¡Comenzar Partida!*", se oculta el modal inicial, arranca el cronómetro desde `00:00.0` y comienza el bucle del juego (`update()` a 60 fotogramas por segundo).
5. **Exploración, Botiquines y Jefes:** A medida que avanzás, `city.js` genera nuevos bloques de edificios y 1 botiquín por bloque (que cura el 100% y reaparece cada 45s). Al llegar al puntaje de un Jefe (ej. 1.000 pts para Kingpin), **dejan de salir enemigos comunes** y el mapa se encierra en **2 pantallas** con barreras láser rojas hasta derrotarlo.
6. **Fin de la partida y Registro:** Si perdés toda la vida, si mantenés apretada la tecla `R` por 3 segundos, o si vencés al jefe final (Doctor Doom a los 40.000 pts), se frena el reloj y aparece el formulario para guardar tu **Nombre y Apellido** en MariaDB.

3. RESUMEN GENERAL DE TECNOLOGÍAS UTILIZADAS

Tecnología / Herramienta	Dónde se usa	Para qué función principal se utilizó
Python 3 (Pillow + NumPy + SciPy)	`tools/slice_sprites.py` `tools/build_hero_assets.py` `tools/make_art.py`	Procesamiento previo de imágenes: recorta automáticamente las hojas de sprites (*sprite sheets*), elimina el color de fondo a transparente, centra los cuadros de Spider-Man en celdas de `64×72 px` y genera texturas pixel-art de edificios y fondos.
HTML5 + CSS3	`frontend/index.html` `frontend/style.css`	Estructura el lienzo del juego (`#game`) y diseña las ventanas emergentes (inicio, ranking y registro final), controles y medallas con estética de cómic de Marvel.
JavaScript (ES6+)	`frontend/src/*.js`	Lenguaje central de toda la lógica del juego: física vectorial del péndulo, máquinas de estados de IA, generación procedural de la ciudad y peticiones asíncronas `fetch()` .
Phaser 3 (v3.80.1) (Motor 2D WebGL/Canvas)	Todo el `frontend/`	Provee el renderizado acelerado por GPU (WebGL), el motor físico **Arcade Physics**, la cámara suave **Camera2D**, las animaciones interpoladas (**Tweens**) y el gestor de texturas.
PHP 8 + PDO	`backend/api/puntajes/` `backend/config/`	Construye la API REST que recibe y devuelve JSON, valida los datos, aplica el tope de 40.000 puntos y ejecuta consultas SQL preparadas contra la base de datos.
SQL (MariaDB) + XAMPP	`database/schema.sql`	Almacena de forma persistente los puntajes en la tabla `puntajes` y resuelve el ranking con `ORDER BY puntos DESC, tiempo_segundos ASC` .

4. ¿QUÉ HACE CADA ARCHIVO DEL PROYECTO? (GUÍA RÁPIDA)

Archivo	¿Qué hace explicado de forma simple?
`frontend/index.html`	Esqueleto web con el contenedor `#game`, la pantalla inicial con la tabla y el modal final de registro.
`frontend/style.css`	Estilos visuales de historieta, bordes rojos, medallas de oro/plata/bronce y diseño adaptable.
`frontend/src/config.js`	Constantes globales: pantalla (`1280×720`), altura de la calle (`STREET: 1400`), gravedad (`1500`) y vida (`100`).
`frontend/src/assets_data.js`	Catálogo del juego: define los 10 enemigos comunes, los 9 jefes (vidas y puntajes) y las rutas de los **440 PNG**.
`frontend/src/textures.js`	Dibuja por código las texturas auxiliares (efectos, ventanas de edificios y el botiquín blanco con cruz roja).
`frontend/src/city.js`	Genera la ciudad infinita en bloques de `1000 px` (*chunks*), coloca los edificios, el suelo y el botiquín cada 45s.
`frontend/src/player.js`	Controla a Spider-Man: correr, saltar, trepar muros, balanceo en péndulo (clic derecho), golpes (clic izq.) y red (`F`).
`frontend/src/enemies.js`	Inteligencia Artificial y animaciones de enemigos y jefes (terrestres y voladores) y ajuste de pies sobre la calle.
`frontend/src/ui.js`	Dibuja el HUD superior en vivo: barra de vida, puntos, reloj, aviso de próximo jefe y barra de reinicio con `R`.
`frontend/src/main.js`	Director general: une todos los módulos, encierra el mapa en 2 pantallas durante los jefes y conecta con PHP.
`backend/config/conexionBaseDatos.php`	Conecta con MariaDB en XAMPP (usuario `root`) y crea automáticamente la BD `spiderman_db` y la tabla `puntajes`.
`backend/config/cabecerasCors.php`	Habilita los permisos HTTP/JSON para que el navegador pueda comunicarse con el servidor PHP sin bloqueos.
`backend/config/funcionesAuxiliares.php`	Funciones de ayuda para leer el cuerpo JSON enviado por el navegador y responder con el código HTTP adecuado.
`backend/api/puntajes/index.php`	Endpoint principal: con `GET` devuelve el Top 20; con `POST` valida nombre, apellido, puntos (máx. 40.000) y tiempo.
`database/schema.sql`	Script SQL con la creación de la base de datos `spiderman_db` y la tabla `puntajes` en español.

5. CÓMO FUNCIONAN LOS SPRITES (DE PYTHON AL JUEGO EN VIVO)

> **¿Qué es un Sprite?** Es una imagen 2D con fondo transparente (`.png`) que muestra una pose de un personaje. Para lograr que camine, salte o ataque, el juego funciona igual que un **dibujo animado o folioscopio (flipbook)**: cambia de imagen muy rápido (a razón de **8 cuadros por segundo**) dando la ilusión de movimiento continuo.

Etapa 1: Preparación y Recorte Automático de Sprites con Python

Python 3 + Pillow (PIL) + NumPy + SciPy

Originalmente, los dibujos de los personajes venían agrupados en grandes láminas llamadas **hojas de sprites (sprite sheets)** con un color de fondo liso. Para no recortar cientos de dibujos a mano, se utilizaron 3 scripts en **Python**:

- **1. Recorte inteligente y transparencia (`tools/slice_sprites.py`):**
  - Abre la hoja de sprites con `PIL.Image` y la convierte en una matriz numérica `RGBA` usando `NumPy`.
  - Toma el píxel de la esquina superior (`a[2,2,:3]`) para detectar el color de fondo y convierte todos los píxeles de ese color en **transparentes** (`(0, 0, 0, 0)`).
  - Usa visión artificial con `scipy.ndimage` (`binary_dilation` + `label` + `find_objects`) para detectar automáticamente el contorno (*bounding box*) de cada dibujo individual, agrupándolos por filas y columnas y guardándolos como archivos PNG sueltos. Con este proceso se extrajeron y organizaron los **440 sprites PNG** de los 10 enemigos y 9 jefes dentro de `frontend/assets/sprites/`.
- **2. Centrado y alineación de Spider-Man (`tools/build_hero_assets.py`):**
  - Toma cada pose recortada de Spider-Man, calcula su centro horizontal con `np.nonzero(alpha > 0)` y la pega dentro de una celda uniforme de `64×72 píxeles`.
  - Si es una pose de suelo (`hero_idle`, `hero_run0..5`, `hero_punch0..2`), apoya los pies exactamente en el borde inferior (`CH - im.height`); si es una pose aérea (`hero_jump`, `hero_swing`, `hero_climb`), la centra a altura `AIR_Y = 48`. Así Spider-Man nunca "tiembla" ni cambia de posición al cambiar de animación.
- **3. Generación de texturas pixel-art de ciudad y respaldo (`tools/make_art.py`):**
  - Usa matrices de `NumPy` y `Pillow` para dibujar por código las texturas repetibles sin costuras de los edificios (`win_tower.png`, `win_brick.png`, `win_ware.png` de `256×256 px`), el cielo, la luna, las capas lejanas y los sprites de respaldo (usados por ejemplo para *Lizard*, que no tenía hoja de sprites propia).

Etapa 2: Cómo Funcionan y se Animan los Sprites Dentro de la Página

Phaser 3 Loader + Texture Manager + Arcade Body

- **1. El Diccionario de Sprites (`SPRITE_MANIFEST` en `assets_data.js`):** Como sería inmanejable escribir a mano las rutas de 440 archivos en medio del código de combate, `assets_data.js` le asigna un "apodo" corto (clave) a cada archivo PNG (ej. `spr_boss_kingpin_walk_0`) y agrupa las listas de cuadros por acción: estar quieto (`idleKeys`), caminar (`walkKeys`), golpear (`atkKeys`), embestir (`chargeKeys`) o recibir daño (`hitKeys`).
- **2. Precarga en Memoria y Nitidez Pixel-Art (`main.js`):** En la función `preload()`, Phaser recorre el diccionario y descarga todas las imágenes en la tarjeta gráfica con `this.load.image(clave, ruta)`. Luego les aplica el filtro `Phaser.Textures.FilterMode.NEAREST` para conservar los bordes nítidos del pixel-art al agrandarlos.
- **3. Animación por Máquina de Estados (`player.js` y `enemies.js`):** En cada fotograma se suma el tiempo transcurrido a un reloj de animación (`d.animT += dt`). Con la fórmula matemática `Math.floor(d.animT * 8) % frames.length`, el juego elige cuál de las fotos de la lista toca mostrar a 8 FPS y actualiza la imagen en pantalla con `setTexture(key)`. Si el personaje mira hacia la izquierda, espeja el dibujo horizontalmente con `setFlipX(true)` sin necesitar archivos duplicados.
- **4. Punto de Apoyo en los Pies para No Hundirse en la Calle (`setOrigin(0.5, 1.0)`):** Como los dibujos de un jefe al caminar o saltar tienen distinto alto en píxeles, si se centraran por la mitad del cuerpo parecería que se hunden en el asfalto. Por eso todos usan `setOrigin(0.5, 1.0)` (centro horizontal, borde inferior en la planta de los pies) y en cada cambio de cuadro recalibran su caja física con `body.setSize()` y `body.setOffset()`, manteniendo los pies siempre clavados sobre la calle (`Y = 1400`).

6. TECNOLOGÍAS APLICADAS A CADA FUNCIONALIDAD DEL JUEGO

A. Movimiento del Personaje y Balanceo en Telaraña

Phaser Arcade Physics + Trigonometría JS (player.js)

- **Carrera, Salto y Gravedad:** Utiliza el cuerpo físico **Arcade Physics** (`this.sp.body`). La gravedad global (`GRAV: 1500`) atrae al personaje hacia abajo, mientras que las teclas `A/D` y `Espacio` modifican directamente el vector de velocidad con `setVelocityX()` y `setVelocityY(-640)`.
- **Trepar y Saltar por Paredes:** El motor físico detecta el contacto lateral con edificios mediante las banderas booleanas `body.blocked.left` y `body.blocked.right`. Si toca un muro en el aire y se presiona `W`, asciende a velocidad constante (`-240 px/s`).
- **Balanceo con Telaraña (Clic Derecho):** Combina la detección del puntero de Phaser (`activePointer.rightButtonDown()`) con **física vectorial de péndulo en JavaScript**: mide la distancia al punto de anclaje del edificio con `Math.hypot(dx, dy)`. Cuando Spider-Man alcanza el largo de la cuerda (`w.len`), restringe su posición a la circunferencia del péndulo y anula la velocidad radial mediante producto escalar, transformando la caída vertical en impulso de balanceo. Además, inclina el sprite con `Math.atan2()`.
- **Esquiva / Dash (Tecla Shift):** Desactiva la gravedad por `0.18s` (`body.allowGravity = false`), aplica un impulso horizontal de `720 px/s` y otorga `0.35s` de invulnerabilidad (`this.inv = 0.35`).

B. Creación del Mapa y Generación del Mapa Infinito

`JavaScript Map + RandomDataGenerator + Graphics (city.js)`

- **División en Bloques (Chunks de 1.000 px):** Para no saturar la memoria RAM, la ciudad se divide en tramos de `1000 px` de ancho llamados *chunks*, almacenados en una estructura `new Map()` de JavaScript.
- **Carga y Liberación Dinámica:** En cada frame se calcula en qué bloque está Spider-Man con `ci = Math.floor(px / 1000)`. El sistema genera siempre 3 bloques por delante (`ci + 3`) y destruye de la memoria RAM (`destroy()`) los bloques que quedaron más de 3 tramos atrás (`i < ci - 3`).
- **Generación Procedural con Semilla:** Usa `Phaser.Math.RandomDataGenerator` con una semilla determinista por bloque para elegir alturas, anchos y estilos de fachadas, farolas y autos estacionados.
- **Piso Físico y Zonas de Enganche:** El suelo de la calle se crea como un cuerpo estático (`physics.add.staticGroup()`) en `Y = 1400`, y los rectángulos de los edificios se guardan en `c.rects` para validar dónde puede adherirse la telaraña.

C. Desplazamiento del Mapa y Efecto de Profundidad (Parallax)

`Phaser Camera2D + TileSprites Parallax (main.js / city.js)`

- **Seguimiento Suave (Lerp):** La cámara principal sigue a Spider-Man mediante `cameras.main.startFollow(this.player.sp, true, 0.09, 0.09)`, aplicando una interpolación suave del 9% por frame.
- **Adelanto Dinámico de Cámara:** Con `setFollowOffset()`, la cámara se adelanta en la dirección de la velocidad para dar mayor visión hacia donde avanza el jugador.
- **Efecto Parallax Multicapa:** Las siluetas de edificios lejanos usan objetos `Phaser.GameObjects.TileSprite` fijos en pantalla (`setScrollFactor(0)`) cuya textura interna se desplaza al `8%` y `20%` de la velocidad de la cámara (`scrollX * 0.08` y `scrollX * 0.2`), generando sensación de profundidad 3D.

D. Sistema de Ataques (Golpes Cuerpo a Cuerpo y Telaraña a Enemigos)

`Geometría de Cajas Físicas + Tweens (player.js / main.js)`

- **Combo Cuerpo a Cuerpo (Clic Izquierdo):** Alterna un combo de 3 golpes (`this.combo = this.combo % 3 + 1`). En lugar de mostrar ningún cuadro gris de impacto, calcula matemáticamente qué enemigos están frente a Spider-Man a menos de `85 px` en X y a menos de `75 px` de distancia vertical entre sus centros físicos (`body.center.y`). El 3º golpe causa más daño y mayor empuje.
- **Telaraña a Distancia (Tecla F):** Convierte la posición del cursor a coordenadas del mundo (`cameras.main.getWorldPoint`) y busca con `Math.hypot()` al enemigo más cercano al puntero dentro de un radio de `550 px`. Lo paraliza durante `2 segundos` (`e.d.stun = 2`), le dibuja la red encima y le aplica velocidad hacia Spider-Man para atraerlo.

E. Comportamiento de los Enemigos y Jefes (Inteligencia Artificial)

`Máquinas de Estados en JS + Trigonometría (enemies.js)`

- **Enemigos Cuerpo a Cuerpo (Duke, Slim, Spike, etc.):** Calculan la dirección hacia el jugador con `Math.sign(p.x - e.x)`, avanzan hasta estar a `~42 px` y golpean respetando su tiempo de recarga (`cd`). Saltan automáticamente al chocar contra obstáculos.
- **Enemigos a Distancia (Blade y Moose):** Si Spider-Man se acerca a menos de `200 px`, retroceden para mantener distancia; a rango medio apuntan con `Math.atan2(dy, dx)` y disparan proyectiles.
- **Jefes Terrestres (Kingpin, Scorpion, Lizard, Electro, Sandman, Venom):** Alternan por tiempo entre 3 estados: `move` (perseguir y atacar), `wind` (preparación de `0.6s` tiñéndose de rojo para avisar el ataque) y `charge` (embestida veloz a `2.4x`; Electro además dispara un abanico de 5 rayos).
- **Jefes Voladores (Hobgoblin, Duende Verde, Doctor Doom):** Desactivan su gravedad (`allowGravity = false`), levitan siguiendo una onda senoidal matemática (`Math.sin(d.flyT) * 45`), lanzan bombas calabaza o ráfagas triples de energía y ejecutan vuelos rasantes en picada.

F. Cura del Botiquín y Reaparición cada 45 Segundos

`Phaser Graphics + Sine Tweens + Delta Time (textures.js / city.js)`

- **Ubicación Fija por Bloque:** La textura del botiquín se genera por código con `Phaser.GameObjects.Graphics`. Al crearse cada bloque de ciudad (*chunk*), se coloca exactamente **1 botiquín en la mitad del bloque** (`medX = x0 + 500`) animado con una levitación suave infinita (`Sine.easeInOut`).
- **Curación al 100% y Cooldown de 45s:** Cuando Spider-Man pasa a menos de `45 px` (`Math.hypot < 45`) y no tiene la vida llena, restaura su salud al **100%** (`p.hp = CFG.PLAYER_HP`), oculta el botiquín y activa un temporizador propio de `45 segundos` (`m.cooldown = 45`) que se descuenta frame a frame (`m.cooldown -= dt`) hasta reaparecer en ese mismo lugar.

G. Limitación del Mapa a 2 Pantallas Durante el Jefe

World Bounds + Camera Bounds + Math.Clamp (main.js / player.js / enemies.js)

- **Cálculo de la Arena (2.560 px):** Como cada pantalla mide `1280 px` de ancho (`CFG.W`), al alcanzar el puntaje de un jefe la función `startBossFight()` calcula una arena cerrada de exactamente 2 pantallas (`arenaW = 2560 px`) centrada en la posición actual del jugador (`minX` y `maxX`).
- **Triple Bloqueo + Barreras + Pausa de Enemigos Comunes:** Fija los límites de la cámara (`cameras.main.setBounds`), activa paredes físicas invisibles (`physics.world.setBounds`) y aplica `Phaser.Math.Clamp(x, minX, maxX)` tanto a Spider-Man como al jefe. Al mismo tiempo dibuja las barreras láser rojas con `Phaser.Graphics`, elimina a los matones comunes en pantalla y frena su aparición hasta que el jefe es derrotado y `endBossFight()` reabre el mapa infinito.